

INHA UNIVERSITY TASHKENT
DEPARTMENT OF CSE & ICE
SPRING SEMESTER 2024

SOC 2040 - SYSTEMS PROGRAMMING
LAB ASSIGNMENT 4

INSTRUCTIONS :

- ALL LAB ASSIGNMENTS ARE TO BE COMPLETED IN GROUPS OF 4 TO 6 STUDENTS.
- LAB ASSIGNMENT REPORT SHOULD BE PREPARED USING THE LAB ASSIGNMENT 4 REPORT TEMPLATE PROVIDED AT THE ECLASS PORTAL. **NO OTHER REPORT FORMAT WILL BE ACCEPTED AND WILL GET A GRADE 0**
- **HARDCOPY OF THE LAB ASSIGNMENT 4 REPORT MUST BE SPIRALLY BOUNDED. ONLY SPIRALLY BOUNDED HARD COPY OF THE REPORT IS ACCEPTED AND ANY OTHER FORM IS REJECTED OUTRIGHTLY.**
- **LAST DATE FOR UPLOADING THE LAB ASSIGNMENT4 REPORT AT THE ECLASS PORTAL IS MONDAY 29TH APRIL 2024.**
- **EVERY MEMBER OF THE TEAM MUST UPLOAD THE SOFTCOPY OF THE REPORT AT THE ECLASS PORTAL.**
- **NO GRADES WILL BE GIVEN IF SOFTCOPY OF THE REPORT IS NOT UPLOADED AT THE ECLASS.**
- **NO EMAIL SUBMISSION IS ACCEPTED – IT WILL BE OUTRIGHTLY REJECTED AND GIVEN A GRADE 0.**

- **ONE HARD COPY OF THE LAB ASSIGNMENT4 REPORT OF EACH GROUP LEADER SHOULD BE HANDED IN AT THE OFFICE BY THE GROUP LEADER ON TUESDAY 30TH APRIL 2024 BETWEEN 1.30PM & 4.30PM.**
- **HARDCOPY REPORT OF ONLY THE LEADER OF THE APPROVED TEAM GIVEN AT THE ECLASS PORTAL WILL BE ACCEPTED AT MY OFFICE TUESDAY 30TH APRIL 2024 BETWEEN 1.30PM & 4.30PM.**
- **PLEASE NOTE THAT SOFTCOPY OF ALL MEMBERS INCLUDING THE TEAM LEADER OF ONLY THE APPROVED TEAM UPLOADED AT THE ECLASS WILL BE CONSIDERED FOR GRADING.**
- **STUDENTS WHOSE NAMES NOT IN THE APPROVED LIST GIVEN AT THE ECLASS PORTAL WILL BE GIVEN GRADE 0.**
- **LATE SUBMISSION WILL NOT BE ACCEPTED AND GIVEN A GRADE 0.**
- **PRINTOUTS CAN BE TAKEN ON BOTH SIDES OF THE PAPER.**
- **PRINTOUTS TAKEN MUST BE LEGIBLE AND PERFECTLY READABLE – OTHERWISE IT WILL NOT BE GRADED AND GIVEN A GRADE 0 OUTRIGHTLY.**
- **FOR QUESTIONS 1 TO 6 IN PART A & PART B, ‘C’ PROGRAMS WRITING CAN BE SHARED AMONG MEMBERS. BUT EVERY MEMBER MUST EXECUTE ALL PROGRAMS DEVELOPED ON HIS/HER COMPUTER SYSTEM AND REQUIRED TO PROVIDE SCREENSHOTS OF PROGRAM ENTERED, INPUTS GIVEN AND RESULTS**
- **ALL FILES CONTAINING ‘C’ PROGRAMS WRITTEN AND EXECUTED FOR ALL QUESTIONS, ALL MATRIX FILES GENERATED IN QUESTION 4 AND ALL MATRIX RESULT FILES GENERATED IN QUESTIONS 5 & 6 SHOULD BE UPLOADED AT THE E-CLASS PORTAL ALONG WITH THE LAB ASSIGNMENT 4 REPORT.**
- **YOU ARE REQUIRED TO PUT ALL THE FILES IN A FOLDER AND GIVE THE FOLDER NAME - YOUR ID FOLLOWED BY SPLABASSIGNMENT4(EXAMPLE:U2210100_SPLABASSIGNMENT4).**
- **LATE SUBMISSIONS ARE NOT ENTERTAINED, ADHERE TO THE DEADLINE STRICTLY.**
- **READ THE QUESTIONS CORRECTLY & CAREFULLY**

ADDITIONAL INSTRUCTIONS REGARDING LAB ASSIGNMENT4 MATRICES

PART-B: QUESTIONS 4, 5 & 6:

YOU ARE REQUIRED TO TAKE SCREENSHOTS OF ALL MATRICES GENERATED ALONG WITH THE PROGRAM ENTERED, COMMAND EXECUTED.

THERE ARE MANY MATRICES WHICH ARE VERY BIG AND RUN INTO SEVERAL PAGES. IN SUCH CASES, YOU NEED TO INCLUDE SCREENSHOTS OF ATLEAST FIRST 5 PAGES OF EVERY MATRIX ELEMENTS GENERATED AND ALSO THE RESULTING MATRICES AFTER THE SPECIFIED OPERATIONS

PLEASE NOTE THAT MATRICES GENERATED AND RESULTING MATRICES OBTAINED ARE TO BE STORED IN SEPARATE DESIGNATED FILES.

- FOR QUESTIONS 1 TO 6 IN PART A & PART B, 'C' PROGRAMS WRITING CAN BE SHARED AMONG MEMBERS. BUT EVERY MEMBER MUST EXECUTE ALL PROGRAMS DEVELOPED ON HIS/HER COMPUTER SYSTEM AND REQUIRED TO PROVIDE SCREENSHOTS OF PROGRAM ENTERED, INPUTS GIVEN AND RESULTS

FOR Q1) LARGER SIZE VECTORS ARE TO BE TAKEN FOR GETTING FULL POINTS. SMALLER SIZE VECTORS WILL NOT GIVE YOU FULL POINTS

SIMILARLY FOR Q2) LARGER SIZE MATRICES ARE TO BE TAKEN FOR GETTING FULL POINTS. SMALLER SIZE MATRICES WILL NOT GIVE YOU FULL POINTS

FOR Q3) n VALUE SHOULD VARY FROM 10 TO 60 IN STEPS OF 10, i.e., n=10, 20, 30, 40, 50, 60

EACH NODE OF THE LINKED LIST MUST CONTAIN PROPER INFORMATION OF THE STUDENTS IN YOUR CLASS. VISIT ECLASS TO GET INFORMATION ABOUT THE STUDENTS FROM ANY LAB ASSIGNMENT APPROVED TEAM LIST.

YOU CANNOT USE ANY ARBITRARY NAMES & VALUES WHICH WILL NOT BE ACCEPTED AND GIVEN A GRADE 0 FOR THAT ARBITRARY INPUTS GIVEN.

FOR EACH VALUE OF n, consider both the cases (i) $n > m$ and (ii) $n = m$.

n VALUES CAN BE SHARED AMONG THE MEMBERS AND ACCORDINGLY SCREENSHOTS CAN BE SHARED AMONG THE MEMBERS

- READ ALL THE QUESTIONS CORRECTLY & CAREFULLY

QUESTIONS

PART-A : LINKING & LIBRARY INTERPOSITIONING

1. Write separate 'C' Library routines to perform the following vector operations:

a) Vector Addition

- (i) **iadd_vec(x, y, z, n)** – to add two vectors x & y containing **n integers** and store the result in vector z i.e., $z[i] = x[i] + y[i]$ for $i = 0, \dots, n-1$
- (ii) **radd_vec(p, q, r, m)** – to add two vectors p & q containing **m real numbers** and store the result in vector r i.e., $r[i] = p[i] + q[i]$ for $i = 0, \dots, m-1$

b) Vector Subtraction

- (i) **isub_vec(x, y, z, n)** – to subtract two vectors x & y containing **n integers** and store the result in vector z i.e., $z[i] = x[i] - y[i]$ for $i = 0, \dots, n-1$
- (ii) **rsub_vec(p, q, r, m)** – to subtract two vectors p & q containing **m real numbers** and store the result in vector r i.e., $r[i] = p[i] - q[i]$ for $i = 0, \dots, m-1$

c) Vector Multiplication

- (i) **imul_vec(x, y, z, n)** – to multiply two vectors x & y containing **n integers** and store the result in vector z i.e., $z[i] = x[i] * y[i]$ for $i = 0, \dots, n-1$
- (ii) **rmul_vec(p, q, r, m)** – to multiply two vectors p & q containing **m real numbers** and store the result in vector r i.e., $r[i] = p[i] * q[i]$ for $i = 0, \dots, m-1$

d) Vector Division

- (i) **idiv_vec(x, y, z, n)** – to divide two vectors x & y containing **n integers** and store the result in vector z i.e., $z[i] = x[i] / y[i]$ for $i = 0, \dots, n-1$
- (ii) **rdiv_vec(p, q, r, m)** – to divide two vectors p & q containing **m real numbers** and store the result in vector r i.e., $r[i] = p[i] / q[i]$ for $i = 0, \dots, m-1$

e) Multiplying a vector by a constant

- (i) **imulk_vec(x, k, z, n)** – to multiply every element in a vector **x** containing **n integers** by an integer constant **k** and store the result in vector **z** i.e., $z[i] = x[i] * k$ for $i = 0, \dots, n-1$
- (ii) **rmulk_vec(p, j, r, m)** – to multiply every element in a vector **p** containing **m real numbers** by a real number constant **j** and store the result in vector **r** i.e., $r[i] = p[i] * j$ for $i = 0, \dots, m-1$

f) Dividing a vector by a constant

- (i) **idivk_vec(x, k, z, n)** – to divide every element in a vector **x** containing **n integers** by an integer constant **k** and store the result in vector **z** i.e., $z[i] = x[i] / k$ for $i = 0, \dots, n-1$
- (ii) **rdivk_vec(p, j, r, m)** – to divide every element in a vector **p** containing **m real numbers** by real number constant **j** and store the result in vector **r** i.e., $r[i] = p[i] / j$ for $i = 0, \dots, m-1$

g) Reversing a vector

- (i) **irev_vec(x, y, n)** - to reverse the vector **x** containing **n integers** elements and store the reversed vector in **y**, i.e., $y[i] = x[(n-1) - i]$ for $i = 0, \dots, n-1$
- (ii) **rrev_vec(p, r, m)** - to reverse the vector **p** containing **m real numbers** elements and store the reversed vector in **r**, i.e., $r[i] = p[(m-1) - i]$ for $i = 0, \dots, m-1$

h) Vector Sorting with a parameter **s** where **s=1** means ascending order & **s=0** means descending order

- (i) **isort_vec(x, s, z, n)** – to sort a vector **x** containing **n integers** in ascending order if **s=1** else descending order if **s=0** and store the result in vector **z** i.e., **$z = \text{sort}(x)$ ascending order if $s=1$ or $z = \text{sort}(x)$ descending order if $s=0$**
- (ii) **rsort_vec(p, s, r, m)** – to sort a vector **p** containing **m real numbers** in ascending order if **s=1** else descending order if **s=0** and store the result in vector **r** i.e., **$r = \text{sort}(p)$ ascending order if $s=1$ or $r = \text{sort}(p)$ descending order if $s=0$**

You are required to create the following :

- (i) Library Archive **libvector.a** for static linking
- (ii) Sharable Library **libvector.so** for dynamic linking

Next, write separate programs to invoke the above written vector library routines to carry out vector addition, vector subtraction, vector multiplication, vector division, vector multiplication by a constant, vector division by constant, Reversing a vector, vector sorting in ascending or descending order as directed by s in the main program for both integers and real numbers.

Perform the linking of required library routines to your object file

- (i) at compile time and execute
- (ii) at Load time and execute
- (iii) at Run time

The size n , Elements of integer vectors x & y , constant k and sort parameter s value are to be read from the keyboard during execution time so that 36 different inputs can be used by taking different values for n , x , y , k & for s , both $s=0$ & $s=1$ for each input and 36 different output vectors z can be generated.

Similarly, the size m , Elements of real number vectors p & q , constant j and sort parameter s value are to be read from the keyboard during execution time so that 36 different inputs can be used by taking different values for m , p , q , j & for s , both $s=0$ & $s=1$ for each input and 36 different output vectors z can be generated.

YOU ARE REQUIRED TO USE LARGER VECTORS(n & m SHOULD BE ≥ 15) TO GET FULL POINTS.

PROVIDE THE ALGORITHM, PROGRAM AND SCREENSHOTS OF PROGRAM, COMMANDS ENTERED, INPUTS GIVEN & RESULTS OBTAINED

EVERY MEMBER MUST EXECUTE ALL PROGRAMS DEVELOPED ON HIS/HER COMPUTER SYSTEM AND REQUIRED TO PROVIDE SCREENSHOTS OF PROGRAM ENTERED, COMMANDS EXECUTED, INPUTS GIVEN AND RESULTS

2. Write separate 'C' Library routines to perform the following matrix operations :

a) Matrix Addition

- (i) **imatadd(a, b, c, n)** – to add two **n x n** matrices a & b containing **integers** and store the result in matrix c i.e., $c = (a + b)$
- (ii) **rmatadd(p, q, r, m)** – to add two **m x m** matrices p & q containing **real numbers** and store the result in matrix r i.e., $r = (p + q)$

b) Matrix Subtraction

- (i) **imatsub(a, b, c, n)** – to subtract two **n x n** matrices a & b containing **integers** and store the result in matrix c i.e., $c = (a - b)$
- (ii) **rmatsub(p, q, r, m)** – to subtract two **m x m** matrices p & q containing **real numbers** and store the result in matrix r i.e., $r = (p - q)$

c) Matrix Multiplication

- (i) **imatmult(a, b, c, n)** – to multiply two **n x n** matrices a & b containing **integers** and store the result in matrix c i.e., $c = (a * b)$
- (ii) **rmatmult(p, q, r, m)** – to multiply two **m x m** matrices p & q containing **real numbers** and store the result in matrix r i.e., $r = (p * q)$

d) Matrix Transpose

- (i) **imattrans(a, c, n)** – to transpose a given **n x n** matrix a containing **integers** and store the result in matrix c i.e., $c = \text{Transpose}(a)$
- (ii) **rmattrans(p, r, m)** – to transpose a given **m x m** matrix p containing **real numbers** and store the result in matrix r i.e., $r = \text{Transpose}(p)$

You are required to create the following :

- (i) Library Archive **libmatrix.a** for static linking
- (ii) Sharable Library **libmatrix.so** for dynamic linking

Next, write separate programs to invoke the above written matrix library routines to carry out matrix operations on the matrices A & B in the main program and **perform the linking of required library routines to your object file**

- (i) at compile time and execute
- (ii) at Load time and execute
- (iii) at Run time

The size n, Elements of integer MATRICES a & b, are to be read from the keyboard during execution time so that 36 different inputs can be used by taking different values for n, matrices a & b and 36 different output matrices c can be generated.

Similarly, the size m, Elements of real number MATRICES p & q, are to be read from the keyboard during execution time so that 36 different inputs can be used by taking different values for m, matrices p & q and 36 different output matrices r can be generated.

PROVIDE THE ALGORITHM, PROGRAM AND SCREENSHOTS OF PROGRAM, COMMANDS ENTERED, INPUTS GIVEN & RESULTS OBTAINED

EVERY MEMBER MUST EXECUTE ALL PROGRAMS DEVELOPED ON HIS/HER COMPUTER SYSTEM AND REQUIRED TO PROVIDE SCREENSHOTS OF PROGRAM ENTERED, COMMANDS EXECUTED, INPUTS GIVEN AND RESULTS

3. Library Inter-positioning – You are required to use inter-positioning to trace the number of calls made to **malloc** and **free** functions as the program runs in order to detect memory leaks in the program and also introduce FIX-UP code to remove memory leaks from the program.

You are required to write separate wrapper functions for **malloc** and **free** to carry out inter-positioning at

- (i) Compile Time
- (ii) Link Time

Write your main program to create a linked list containing 'n' nodes containing student information(such as STUDENT NAME, STUDENT ID, SECTION NO, PROGRAM (CSE/ICE), LEVEL(SOPHOMORE), EMAIL, PHONE NO.) in every node in such a way that it MAKES 'n' number of calls to malloc TO CREATE 'n' NODES OF THE LINKED LIST and THEN MAKES 'm' number of calls to free SOME NODES to delete student information, where case (i) is $n > m$ and case (ii) is $n=m$. Consider all cases to show that memory leaks are detected and resolved. Provide suitable outputs.

EVERY MEMBER MUST EXECUTE THE PROGRAM FOR DIFFERENT LARGER VALUES OF n & m.

**n VALUE SHOULD VARY FROM 10 TO 60 IN STEPS OF 10, i.e.,
n=10, 20, 30, 40, 50, 60**

FOR EACH VALUE OF n, consider both the cases (i) $n > m$ and (ii) $n=m$.

n VALUES CAN BE SHARED AMONG THE MEMBERS AND ACCORDINGLY SCREENSHOTS CAN BE SHARED AMONG THE MEMBERS

PROVIDE THE ALGORITHM, PROGRAM AND SCREENSHOTS OF PROGRAM, COMMANDS ENTERED, INPUTS GIVEN & RESULTS OBTAINED

EVERY MEMBER MUST EXECUTE ALL PROGRAMS DEVELOPED ON HIS/HER COMPUTER SYSTEM AND REQUIRED TO PROVIDE SCREENSHOTS OF PROGRAM ENTERED, COMMANDS EXECUTED, INPUTS GIVEN AND RESULTS

PART-B :MATRIX MULTIPLICATION & EXPLOITING CACHE LOCALITY

4. Given a **matgen2file.c** file containing 'C' program to generate a N x N matrix of real (double) elements where N can take any value between 10 & 1000 (N = 10, 20, 40, 60, 80, 100, 200, ..., 900, 1K(K=1000)) or even more. This program initializes the values of the elements of the matrix with real numbers using functions – **srand** to initialize the random seed and **rand()** to generate the random number.

Compile the program **matgen2file.c** to create the executable file using the following command :

```
$gcc -o matgen2file matgen2file.c
```

This program includes command line argument to read the value of N and filename entered as arguments along with the command **matgen2file** (filename is the second command line argument and N is the third command line argument after the command **matgen2file**) as given below:

For example :

If **matgen2file** is the executable file name then the command line arguments to execute your program to generate and store a 400 x 400 matrix A (where N=400) in a file by name **mat400x400A** will be

```
$ ./matgen2file mat400x400A 400
```

You are required to run the program for different values of N, where N can take any value between 10 & 1000 (N = 10, 20, 40, 60, 80, 100, 200, ..., 900, 1K(K=1000)) or even more. But you are required to store the matrices generated in different files by giving the names for the matrix file as in the command given above (**mat10x10A, mat10x10B, mat20x20A, mat20x20B, mat30x30A, mat30x30B, mat40x40A, mat40x40B, mat50x50A, mat50x50B, mat60x60A, mat60x60B, mat70x70A, mat70x70B, mat80x80A, mat80x80B, mat90x90A, mat90x90B, matmat100x100A, matmat100x100B, mat200x200A, mat200x200B, mat300x300A, mat300x300B, mat400x400A, mat400x400B, mat500x500A, mat500x500B, mat600x600A, mat600x600B, mat700x700A, mat700x700B, mat800x800A, mat800x800B, mat900x900A, mat900x900B, mat1000x1000A, mat1000x1000A,**).

PLEASE NOTE THAT THESE MATRIX FILES A & B IN EACH CASE ARE REQUIRED IN QUESTION 5 TO PERFORM MATRIX MULTIPLICATION

PROGRAM FILE NAME : matgen2file.c

```
/* ***** */
/* C PROGRAM TO GENERATE MATRIX OF NXN SIZE - PROGRAM NAME : matgen2file.c */
/* THIS PROGRAM GENERATES MATRIX ELEMENTS OF TYPE FLOAT OF SIZE N X N */
/* USING THE RANDOM GENERATOR FUNCTION rand() TO GENERATE REAL RANDOM */
/* NUMBERS BETWEEN 0 AND 1000 USING THE FOLLOWING EXPRESSION */
/* ((float) rand() / (float) (RAND_MAX))*1000.0; */
/* */
/* COMPILE THE PROGRAM USING THE FOLLOWING COMMAND */
/* */
/* $ gcc -o matgen2file matgen2file.c */
/* */
/* EXECUTE THE PROGRAM USING THE FOLLOWING COMMAND WHICH REQUIRES */
/* THREE COMMAND LINE ARGUMENTS AS FOLLOWS : */
/* */
/* $ ./matgen2file matrixfilename N */
/* */
/* FOR EXAMPLE: TO CREATE MATRIX A(FILE NAME: mat100x100A) OF SIZE 100X100 */
/* USE THE FOLLOWING COMMAND */
/* */
/* $ ./matgen2file mat100x100A 100 */
/* */
/* THIS COMMAND NOT ONLY GENERATES MATRIX A AND STORES IN FILE */
/* mat100x100A BUT ALSO OUTPUTS CPU TIME (IN SECS) TAKEN TO */
/* GENERATE THE FILE */
/* */
/* ***** */
```

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <string.h>
#include <float.h>
#include <math.h>
```

```
clock_t clock(void);
```

```
int main(int argc, char *argv[])
{
```

```
    FILE *fp1;
    int i, n, nxn;
    float lr = 1000.0, mel;
    double total_t;
    clock_t start_t, end_t;
```

```
    start_t = clock();
```

```
    if (argc == 3)
    {
```

```
        printf("START TIME - TIME BEFORE THE START OF FILE CREATION = %lf clock
cycles\n", (double)start_t);
```

```
        fp1 = fopen(argv[1], "w");
```

```
        n = atoi(argv[2]);
        nxn = n*n;
```

```
        /* Intializes random number generator */
        srand((unsigned int) time(NULL));
```

```

    /* Print nXn random numbers */
    printf(" START OF FILE %s CREATION :\n", argv[1]);
    printf("-----\n");

    for( i = 0; i < nxn; i++ )
    {
        mel= ((float) rand() / (float) (RAND_MAX))*lr;
        fprintf(fp1, "%f ",mel);
    }

    fclose(fp1);

    end_t = clock();
    printf(" END OF FILE %s CREATION :\n", argv[1]);

    printf("END TIME - TIME AFTER THE END OF FILE CREATION = %lf clock cycles\n", (double)end_t);

    printf("CLOCKS_PER_SEC = %ld\n", CLOCKS_PER_SEC);

    total_t = (double)((double)end_t - (double)start_t) / (double)CLOCKS_PER_SEC;

    printf("TOTAL TIME TAKEN BY CPU FOR %s FILE CREATION: %lf secs\n", argv[1], total_t);

    printf("-----\n");
    printf(" ELEMENTS OF %d x %d MATRIX %s : \n", n, n, argv[1]);
    printf("-----\n");
    fp1 = fopen(argv[1], "r");

    while( !feof(fp1))
    {
        fscanf(fp1, "%f ", &mel);
        printf("%f ", mel);
    }
    printf("\n\n");
    printf("-----\n");
    fclose(fp1);
}
else
{
    printf("wrong arguments - use the format \n $matgen2file filename N\n");
}
}

```

PROVIDE THE ALGORITHM, PROGRAM AND SCREENSHOTS OF PROGRAM, COMMANDS ENTERED, INPUTS GIVEN & RESULTS OBTAINED

EVERY MEMBER MUST EXECUTE ALL PROGRAMS DEVELOPED ON HIS/HER COMPUTER SYSTEM AND REQUIRED TO PROVIDE SCREENSHOTS OF PROGRAM ENTERED, COMMANDS EXECUTED, INPUTS GIVEN AND RESULT

5. Write **SIX versions of Matrix Multiplication 'C' functions** where each version is uniquely identified by the ordering of its loops. Assign the following function names to each version of the matrix multiplication routine written – **mmatijk, mmatjik, mmatjki, mmatkji, mmatkij, mmatikj**.
- (i) For all the above six versions of the matrix multiplication routines, you are required to write a main program in which you will be initially reading the value of N and two matrix elements (call them A & B) from files already created in Question 4 and compute the product matrix (call it matrix C) using each matrix multiply version (six versions) you have developed and store the resultant matrix C elements in six separate files (one for each version) for each value of N. **N can be varied from 100 to 1000 in steps of 100, i.e, N=100, 200, 300, 400, 500, 600, 700, 800, 900 & 1000. (You can the read value of N, filenames of Matrices A, B & C from the keyboard as command-line arguments).**
USE MATRICES A & B GENERATED IN QUESTION 4
- (ii) You are also required to note the CPU time (**use clock() function declared in time.h**) taken before (**start_time=clock()**) and after (**end_time=clock()**) each function call so that the time taken by the CPU to execute each function can be computed by taking the difference between **end_time & start_time**. **Store these CPU time taken values in a separate file for each version for different values of N. N can be varied from 100 to 1000 in steps of 100, i. e, N=100, 200, 300, 400, 500, 600, 700, 800, 900 & 1000.**
USE MATRICES A & B GENERATED IN QUESTION 4
- (iii) Finally, you are required to draw a graph (use Excel) showing Time taken by CPU for various values of N for each version of your matrix multiply function and clearly indicate which function takes least amount of CPU time and speedup of that function over others.

(Please note that C library function clock() returns the number of clock ticks elapsed since the program was launched. To get the number of secs used by the CPU, divide this value by CLOCKS_PER_SEC (normally CLOCKS_PER_SEC=1000000))

PROVIDE THE ALGORITHM, PROGRAM AND SCREENSHOTS OF PROGRAM, INPUTS GIVEN & RESULTS OBTAINED

EVERY MEMBER MUST EXECUTE ALL PROGRAMS DEVELOPED ON HIS/HER COMPUTER SYSTEM AND REQUIRED TO PROVIDE SCREENSHOTS OF PROGRAM ENTERED, COMMANDS EXECUTED, INPUTS GIVEN AND RESULTS

6. In this exercise, you are required to multiply matrix A and B, by transposing the second matrix B in the memory after reading it from the file to provide a better Data Locality so that the Row elements of matrices A and B (row elements of B now represent original column elements) can be considered to generate product matrix C elements. i.e., $C[i][j] = A[i][j] * B[i][j]$ for $i=0, \dots, n-1, j=0, \dots, n-1$, i.e., multiplying row elements of Matrix A by row elements of Transposed Matrix B.

Compute the time taken by this version of your Matrix Multiplication program. Repeat this for different values of N from 100 to 1K.

Store the product matrix C elements in a separate file for each N and also the time taken by the CPU to compute the product matrix C for different values of N in a separate file.

N can be varied from 100 to 1000 in steps of 100, i.e, N=100, 200, 300, 400, 500, 600, 700, 800, 900 & 1000.

(You can read value of N, filenames of Matrices A, B & C from the keyboard as command-line arguments).

USE MATRICES A & B GENERATED IN QUESTION 4

PROVIDE THE ALGORITHM, PROGRAM AND SCREENSHOTS OF PROGRAM, COMMANDS ENTERED, INPUTS GIVEN & RESULTS OBTAINED

EVERY MEMBER MUST EXECUTE ALL PROGRAMS DEVELOPED ON HIS/HER COMPUTER SYSTEM AND REQUIRED TO PROVIDE SCREENSHOTS OF PROGRAM ENTERED, COMMANDS EXECUTED, INPUTS GIVEN AND RESULTS
